FINAL PROJECT HANDBOOK

AI-Assisted Laravel Development Project

Student Information Management System with Web & REST API Components

Course: Laravel Web Development
Duration: 2 Weeks
Project Type: Individual Project
Submission: Source Code + Documentation Portfolio + Demonstration (instructor's discretion)
Total Weight: 100% (+10 Bonus)

---

1. PROJECT OVERVIEW

The final project serves as the culminating activity for this course and is designed to assess your mastery of all topics covered throughout Modules 1–10.

Unlike traditional programming projects, this project embraces modern software development practices by allowing and encouraging the use of Artificial Intelligence (AI) tools and agentic development workflows.

You will build a complete **Student Information Management System (SIMS)** using Laravel and demonstrate not only technical implementation skills but also your ability to effectively collaborate with AI systems as a software engineer.

The project evaluation will focus on:

- Laravel Development Skills
- Software Engineering Practices
- AI-Assisted Development
- Prompt Engineering
- Agent Design
- Critical Evaluation of AI Outputs
- Documentation and Reflection

---

2. PROJECT OBJECTIVES

By completing this project, students should be able to:

Laravel Competencies

- Develop applications using MVC Architecture
- Create routes and controllers
- Build forms and process requests
- Design Blade templates
- Implement CRUD operations
- Use migrations and Eloquent ORM
- Manage authentication and authorization
- Consume and create REST APIs

AI Engineering Competencies

- Design effective prompts
- Utilize AI-assisted workflows
- Create specialized AI agents
- Manage context windows and token usage
- Evaluate AI-generated outputs critically
- Document AI-assisted development processes

---

3. PROJECT TITLE

Student Information Management System (SIMS)

Students must develop a web-based Student Information Management System featuring:

Authentication

- User Registration
- Login
- Logout

Student Management

- Add Student
- View Students
- Edit Student
- Delete Student

Subject Management

- Add Subject
- View Subjects
- Edit Subject

- Delete Subject

Enrollment Management

- Enroll Student
- View Enrollments
- Update Enrollment
- Delete Enrollment

Dashboard

- Student Statistics
- Enrollment Statistics
- System Summary

REST API

Students must expose selected resources through REST API endpoints.

---

4. REQUIRED FEATURES

A. Authentication

Must include:

✅ Registration

✅ Login

✅ Logout

B. Authorization

Must include:

✅ Protected Routes

✅ Role-based Access Control

Suggested Roles:

```
Administrator
Staff
```

Administrators should have full access.

Staff may have restricted privileges.

C. Student Module

Required CRUD Operations:

```
Create Student
Read Student
Update Student
Delete Student
```

Suggested Fields:

```
Student Number
First Name
Last Name
Email
Course
Year Level
```

D. Subject Module

Required CRUD Operations:

```
Create Subject
Read Subject
Update Subject
Delete Subject
```

Suggested Fields:

```
Subject Code
Subject Name
Units
```

E. Enrollment Module

Required CRUD Operations:

```
Enroll Student
View Enrollment
Update Enrollment
Delete Enrollment
```

Suggested Relationships:

```
Student
  ⇔
Enrollment
  ⇔
Subject
```

F. Validation

Implement proper validation for:

```
Required Fields
Email Formats
Unique Entries
Input Length Limits
```

G. REST API

Must provide:

Student Endpoints

```
GET     /api/students

GET     /api/students/{id}

POST    /api/students

PUT     /api/students/{id}

DELETE  /api/students/{id}
```

Students may add additional endpoints.

---

5. AI USAGE POLICY

AI IS ALLOWED

The use of AI tools is encouraged.

Examples include:

```
ChatGPT
```

```
Copilot
Claude
Gemini
Cursor
Windsurf
Cline
OpenHands
Roo Code
 Continue.dev
```

However:

AI DOES NOT REPLACE ENGINEERING

Students remain responsible for:

- Code correctness
- Security
- Architecture
- Testing
- Documentation

You must understand and defend every line of submitted code.

---

6. REQUIRED AI ENGINEERING PORTFOLIO

Submit:

```
AI_PORTFOLIO.pdf
```

This document should contain:

A. AI Platform Used

Examples:

```
ChatGPT
Claude
GitHub Copilot
Gemini
Cursor
```

For each platform:

Describe:

- Purpose
- Strengths
- Why it was selected

B. Prompt History

Provide examples of:

Initial Prompt

Example:

```
Create Laravel migration for student management.
```

Refined Prompt

Example:

```
Create Laravel migration for a student management system using best Laravel practices. Include unique student numbers and timestamps.
```

Debugging Prompt

Example:

```
Identify why this Laravel form returns a 419 error.
```

C. Prompt Iterations

Show how prompts evolved.

Demonstrate engineering thinking rather than simple prompt usage.

---

7. AGENT DESIGN REQUIREMENT

Submit:

```
AGENT_DESIGN.pdf
```

Objective

Demonstrate how AI agents were organized to complete the project.

Example Architecture

```
Student Developer
       │
       ▼

Planner Agent
       │

       ├── Database Agent

        ├── Laravel Agent

       ├── Testing Agent

        └── Documentation Agent
```

Example Responsibilities

Planner Agent

Responsible for:

```
Requirements Analysis
Work Breakdown
Project Planning
```

Database Agent

Responsible for:

```
Migrations
Relationships
Schema Design
```

Laravel Agent

Responsible for:

```
Controllers
CRUD Operations
```

```
3  Routes
```

Testing Agent

Responsible for:

```
1  Bug Analysis
2  Validation Review
3  Security Review
```

Documentation Agent

Responsible for:

```
1  README
2  API Documentation
3  Reports
```

8. TOKEN AND CONTEXT MANAGEMENT

Students must explain:

How context was provided

Examples:

```
1  Single File Context
2
3  Feature-Based Context
4
5  Module-Based Context
```

Token Optimization Strategies

Examples:

```
1  Smaller Prompts
2
3  Task Decomposition
4
5  Agent Specialization
6
7  Incremental Generation
```

Reflection Questions

Answer:

1. How did you reduce unnecessary AI context?
2. How did you avoid prompt repetition?
3. How did you verify generated outputs?
4. How did you balance token cost and quality?

---

9. SUBMISSION REQUIREMENTS

Submit:

---

1. Laravel Project

```
ZIP FILE
```

Include:

```
Source Code
README
Database Export
```

Do NOT include:

```
vendor/
node_modules/
```

---

2. AI Portfolio

```
AI_PORTFOLIO.pdf
```

---

3. Agent Design Document

```
AGENT_DESIGN.pdf
```

---

4. Reflection Report

```
REFLECTION.pdf
```

---

5. API Documentation

Suggested:

```
Postman Collection
Swagger/OpenAPI
```

10. PROJECT RUBRIC

A. Laravel Technical Implementation (40%)

Criteria	Weight
Authentication	5%
Authorization	5%
CRUD Functionality	10%
Database Design & Eloquent	10%
REST API Implementation	10%

B. Software Engineering Quality (15%)

Criteria	Weight
MVC Compliance	5%
Code Quality	5%
Project Structure	5%

C. AI Engineering Competency (25%)

Criteria	Weight
Prompt Engineering	10%
Agent Design	10%
Token & Context Management	5%

D. Testing and Validation (10%)

Criteria	Weight
Validation & Error Handling	5%
AI Output Verification	5%

E. Documentation and Reflection (10%)

Criteria	Weight
Technical Documentation	5%

Reflection Report	5%

TOTAL

Category	Weight
Technical Implementation	40%
Software Engineering	15%
AI Engineering	25%
Testing & Validation	10%
Documentation & Reflection	10%
TOTAL	**100%**

11. BONUS POINTS (+10%)

Multi-Agent Workflow

+3 Points

Demonstrate specialized agents.

AI Comparison Study

+2 Points

Compare results from:

```
ChatGPT
Claude
Gemini
Copilot
```

API Documentation

+1 Point

Professional API documentation.

Automated Testing

+2 Points

Laravel Feature Tests or Unit Tests.

Deployment

+2 Points

Successfully deployed online.

Examples:

```
Railway
Render
Laravel Cloud
DigitalOcean
AWS
```

12. ORAL DEFENSE

Students may be required to defend their project. (Instructor's discretion)

Sample questions include:

Laravel

- Explain your MVC implementation.
- Why did you choose your database structure?
- How did Route Model Binding help your project?

AI Engineering

- Why did you choose your AI platform?
- How did your prompts evolve?
- Describe your agent architecture.
- What AI-generated code did you reject and why?

Software Engineering

- What would you improve if given two additional weeks?
- What was the most challenging technical problem you solved?

13. SUCCESS CRITERIA

An outstanding submission demonstrates:

✅ Working Laravel Application

✅ Authentication and Authorization

✅ Full CRUD Functionality

✅ Proper Database Design

✅ REST APIs

✅ Professional Documentation

✅ Well-Engineered Prompts

✅ Clearly Defined AI Agents

✅ Evidence of Critical Thinking

✅ Ability to Explain and Defend Engineering Decisions

FINAL REMINDER

This project is not a test of how much code an AI can generate.

It is a test of your ability to:

- Design software solutions
- Direct AI effectively
- Evaluate AI output critically
- Apply Laravel best practices
- Think like a software engineer

The highest marks will be awarded to students who demonstrate sound engineering judgment, thoughtful AI collaboration, and a deep understanding of the system they build.